

lezione-21-02-24

September 1, 2024

#

WebScraping

Anno Accademico 2022-2023

Docente: Laura Ricci

Lezione 7

Pandas: Filetering and Grouping, Group_By

21 Febbraio 2024

0.1 Assignment 4: Correzione

- considerare il **Data Set** dei giocatori di basket
- indicare due modi diversi per impostare l'indice del DataFrame al nome del giocatore
- calcolare il numero di giocatori per ogni Team
- chi sono i 5 giocatori più pagati?
- chi è il giocatore più vecchio dei New York Knicks?

0.2 Importare il DataFrame

```
[1]: import warnings
warnings.filterwarnings('ignore')

import pandas as pd
pd.read_csv("DataSets/nba.csv", parse_dates = ["Birthday"])
```

```
[1]:
```

	Name	Team	Position	Birthday	Salary
0	Shake Milton	Philadelphia 76ers	SG	1996-09-26	1445697
1	Christian Wood	Detroit Pistons	PF	1995-09-27	1645357
2	PJ Washington	Charlotte Hornets	PF	1998-08-23	3831840
3	Derrick Rose	Detroit Pistons	PG	1988-10-04	7317074
4	Marial Shayok	Philadelphia 76ers	G	1995-07-26	79568
..	...	...	...	...	...
445	Austin Rivers	Houston Rockets	PG	1992-08-01	2174310
446	Harry Giles	Sacramento Kings	PF	1998-04-22	2578800
447	Robin Lopez	Milwaukee Bucks	C	1988-04-01	4767000
448	Collin Sexton	Cleveland Cavaliers	PG	1999-01-04	4764960

```
449      Ricky Rubio           Phoenix Suns      PG 1990-10-21  16200000
```

```
[450 rows x 5 columns]
```

0.3 Indicare due modi diversi per impostare l'indice del DataFrame con il nome del giocatore

```
[2]: nba = pd.read_csv(
      "DataSets/nba.csv", index_col = "Name", parse_dates = ["Birthday"]
    )
nba.head()
```

```
[2]:
```

	Name	Team	Position	Birthday	Salary
	Shake Milton	Philadelphia 76ers	SG	1996-09-26	1445697
	Christian Wood	Detroit Pistons	PF	1995-09-27	1645357
	PJ Washington	Charlotte Hornets	PF	1998-08-23	3831840
	Derrick Rose	Detroit Pistons	PG	1988-10-04	7317074
	Marial Shayok	Philadelphia 76ers	G	1995-07-26	79568

0.4 Indicare due modi diversi per impostare l'indice del DataFrame con il nome del giocatore

```
[3]: nba=nba.setIndex("Name")
```

```
-----
AttributeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_19416\3523637788.py in ?()
----> 1 nba=nba.setIndex("Name")

~\AppData\Local\Programs\Python\Python312\Lib\site-packages\pandas\core\generic
.py in ?(self, name)
    6289         and name not in self._accessors
    6290         and self._info_axis.
-> _can_hold_identifiers_and_holds_name(name)
    6291     ):
    6292         return self[name]
-> 6293     return object.__getattr__(self, name)

AttributeError: 'DataFrame' object has no attribute 'setIndex'
```

- perchè viene sollevato un errore?

0.5 Indicare due modi diversi per impostare l'indice del DataFrame con il nome del giocatore

```
[4]: nba.head()
nba_rs=nba.reset_index()
nba_rs.head()
```

```
[4]:
```

	Name	Team	Position	Birthday	Salary
0	Shake Milton	Philadelphia 76ers	SG	1996-09-26	1445697
1	Christian Wood	Detroit Pistons	PF	1995-09-27	1645357
2	PJ Washington	Charlotte Hornets	PF	1998-08-23	3831840
3	Derrick Rose	Detroit Pistons	PG	1988-10-04	7317074
4	Marial Shayok	Philadelphia 76ers	G	1995-07-26	79568

0.6 Indicare due modi diversi per impostare l'indice del DataFrame con il nome del giocatore

```
[5]: nba_rs=nba_rs.set_index("Name")
nba_rs
```

```
[5]:
```

	Name	Team	Position	Birthday	Salary
	Shake Milton	Philadelphia 76ers	SG	1996-09-26	1445697
	Christian Wood	Detroit Pistons	PF	1995-09-27	1645357
	PJ Washington	Charlotte Hornets	PF	1998-08-23	3831840
	Derrick Rose	Detroit Pistons	PG	1988-10-04	7317074
	Marial Shayok	Philadelphia 76ers	G	1995-07-26	79568
...	...	...	...	...	...
	Austin Rivers	Houston Rockets	PG	1992-08-01	2174310
	Harry Giles	Sacramento Kings	PF	1998-04-22	2578800
	Robin Lopez	Milwaukee Bucks	C	1988-04-01	4767000
	Collin Sexton	Cleveland Cavaliers	PG	1999-01-04	4764960
	Ricky Rubio	Phoenix Suns	PG	1990-10-21	16200000

[450 rows x 4 columns]

0.7 Calcolare il numero di giocatori per ogni Team

```
[6]: nba.Team.value_counts()
```

```
[6]: Team
Philadelphia 76ers      17
Minnesota Timberwolves 17
Utah Jazz               17
Brooklyn Nets           17
Toronto Raptors         16
Boston Celtics          16
```

Chicago Bulls	16
Detroit Pistons	16
New York Knicks	16
New Orleans Pelicans	16
Los Angeles Lakers	16
Charlotte Hornets	16
Atlanta Hawks	15
Denver Nuggets	15
Memphis Grizzlies	15
Phoenix Suns	15
Sacramento Kings	15
Miami Heat	15
Golden State Warriors	15
Los Angeles Clippers	14
Washington Wizards	14
Portland Trail Blazers	14
Milwaukee Bucks	14
Houston Rockets	14
San Antonio Spurs	14
Dallas Mavericks	13
Oklahoma City Thunder	13
Orlando Magic	13
Cleveland Cavaliers	13
Indiana Pacers	13

Name: count, dtype: int64

0.8 Calcolare il numero di giocatori per ogni Team

```
[7]: nba["Team"].value_counts()
```

```
[7]: Team
Philadelphia 76ers      17
Minnesota Timberwolves 17
Utah Jazz              17
Brooklyn Nets          17
Toronto Raptors        16
Boston Celtics         16
Chicago Bulls          16
Detroit Pistons        16
New York Knicks        16
New Orleans Pelicans   16
Los Angeles Lakers     16
Charlotte Hornets      16
Atlanta Hawks          15
Denver Nuggets         15
Memphis Grizzlies      15
Phoenix Suns           15
```

```

Sacramento Kings      15
Miami Heat            15
Golden State Warriors 15
Los Angeles Clippers  14
Washington Wizards    14
Portland Trail Blazers 14
Milwaukee Bucks       14
Houston Rockets       14
San Antonio Spurs     14
Dallas Mavericks      13
Oklahoma City Thunder 13
Orlando Magic         13
Cleveland Cavaliers   13
Indiana Pacers        13
Name: count, dtype: int64

```

0.9 Chi sono i 5 giocatori più pagati?

```
[8]: nba.sort_values("Salary", ascending = False).head()
```

```
[8]:
```

	Team	Position	Birthday	Salary
Name				
Stephen Curry	Golden State Warriors	PG	1988-03-14	40231758
Russell Westbrook	Houston Rockets	PG	1988-11-12	38506482
Chris Paul	Oklahoma City Thunder	PG	1985-05-06	38506482
John Wall	Washington Wizards	PG	1990-09-06	38199000
James Harden	Houston Rockets	PG	1989-08-26	38199000

Chi è il giocatore più vecchio dei New York Knicks?

```
[9]: nba = nba.reset_index().set_index(keys = "Team")
nba.head()
```

```
[9]:
```

	Name	Position	Birthday	Salary
Team				
Philadelphia 76ers	Shake Milton	SG	1996-09-26	1445697
Detroit Pistons	Christian Wood	PF	1995-09-27	1645357
Charlotte Hornets	PJ Washington	PF	1998-08-23	3831840
Detroit Pistons	Derrick Rose	PG	1988-10-04	7317074
Philadelphia 76ers	Marial Shayok	G	1995-07-26	79568

Chi è il giocatore più vecchio dei New York Knicks?

```
[10]: nba_nyk=nba.loc["New York Knicks"].head()
nba_nyk
```

```
[10]:
```

	Name	Position	Birthday	Salary
Team				
New York Knicks	Marcus Morris	PF	1989-09-02	15000000
New York Knicks	Damyeon Dotson	SG	1994-05-06	1618520
New York Knicks	Ignas Brazdeikis	SF	1999-01-08	898310
New York Knicks	Ivan Rabb	PF	1997-02-04	79568
New York Knicks	Kevin Knox	PF	1999-08-11	4380120

```
[11]: nba_nyk.loc["New York Knicks"].sort_values("Birthday").head(1)
```

```
[11]:
```

	Name	Position	Birthday	Salary
Team				
New York Knicks	Marcus Morris	PF	1989-09-02	15000000

0.10 Dove stanno i miei data sets?

```
[12]: import os
print(os.getcwd())
print()
print(os.listdir(os.getcwd()))
print(os.listdir)
try:
    os.chdir('DataSet')
    print(os.getcwd())
    print(os.listdir)
except OSError:
    print("La directory non esiste")
```

C:\Users\ricci\Desktop\Web scraping2324\Lezione_21_02_24\Lezione_21_02_24

['.ipynb_checkpoints', 'DataSets', 'Figures', 'Lezione_21_02_24.ipynb']

<built-in function listdir>

La directory non esiste

- stampa “la directory non esiste”, perchè in realtà il nome della Directory è **DataSets**

0.11 L'Iris DataSet

0.12 L'Iris DataSet

```
[14]: import pandas as pd
from pandas.plotting import scatter_matrix
import matplotlib.pyplot as plt
dataframe = pd.read_csv('DataSets/IrisDataSet.csv')
dataframe
```

```
[14]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa

1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

[150 rows x 5 columns]

0.13 L'Iris Data Set: filtering

- in precedenza abbiamo visto come estrarre righe, colonne e celle da un *DataFrame* utilizzando le funzioni *loc* ed *iloc*
- utili se si conoscono gli indici(label o posizioni) di righe colonne e elementi da selezionari
- **filtering**: selezionare righe mediante una condizione: ad esempio tutte le righe in cui il valore di una colonna verifica una condizione

0.14 L'Iris Data Set: filtering

```
[15]: sp=dataframe["species"]
      sp
```

```
[15]: 0      setosa
      1      setosa
      2      setosa
      3      setosa
      4      setosa
      ...
      145    virginica
      146    virginica
      147    virginica
      148    virginica
      149    virginica
      Name: species, Length: 150, dtype: object
```

- abbiamo selezionato una colonna del *DataFrame*, e il risultato restituito è una *Series*

0.15 L'Iris Data Set: filtering

```
[16]: sp=="setosa"
```

```
[16]: 0      True
      1      True
```

```

2         True
3         True
4         True
...
145      False
146      False
147      False
148      False
149      False
Name: species, Length: 150, dtype: bool

```

- la sintassi sembra errata: come posso confrontare una intera *Series* con un unico valore di tipo *String*?
- *Pandas* capisce che si vuole valutare l'uguaglianza di ogni membro della *series* con la stringa, non l'intera *Series* !
- si confrontano tutti i valori della colonna **species** con il valore **setosa**
- il risultato è una serie di **booleani**

0.16 L'Iris Data Set: filtering

- *Pandas* offre un modo semplice che, data una serie di **booleani**, consente di estrarre tutte le righe che corrispondono al valore **true**
- è sufficiente passare il valore della serie booleana tra parentesi quadrate dopo il nome della serie
- la serie booleana è ottenuta dal confronto della colonna *species* con la stringa *setosa*

```
[17]: dataframe[dataframe["species"] == "setosa"]
```

```

[17]:   sepal_length  sepal_width  petal_length  petal_width  species
0         5.1         3.5         1.4         0.2   setosa
1         4.9         3.0         1.4         0.2   setosa
2         4.7         3.2         1.3         0.2   setosa
3         4.6         3.1         1.5         0.2   setosa
4         5.0         3.6         1.4         0.2   setosa
5         5.4         3.9         1.7         0.4   setosa
6         4.6         3.4         1.4         0.3   setosa
7         5.0         3.4         1.5         0.2   setosa
8         4.4         2.9         1.4         0.2   setosa
9         4.9         3.1         1.5         0.1   setosa
10        5.4         3.7         1.5         0.2   setosa
11        4.8         3.4         1.6         0.2   setosa
12        4.8         3.0         1.4         0.1   setosa
13        4.3         3.0         1.1         0.1   setosa
14        5.8         4.0         1.2         0.2   setosa
15        5.7         4.4         1.5         0.4   setosa
16        5.4         3.9         1.3         0.4   setosa
17        5.1         3.5         1.4         0.3   setosa
18        5.7         3.8         1.7         0.3   setosa

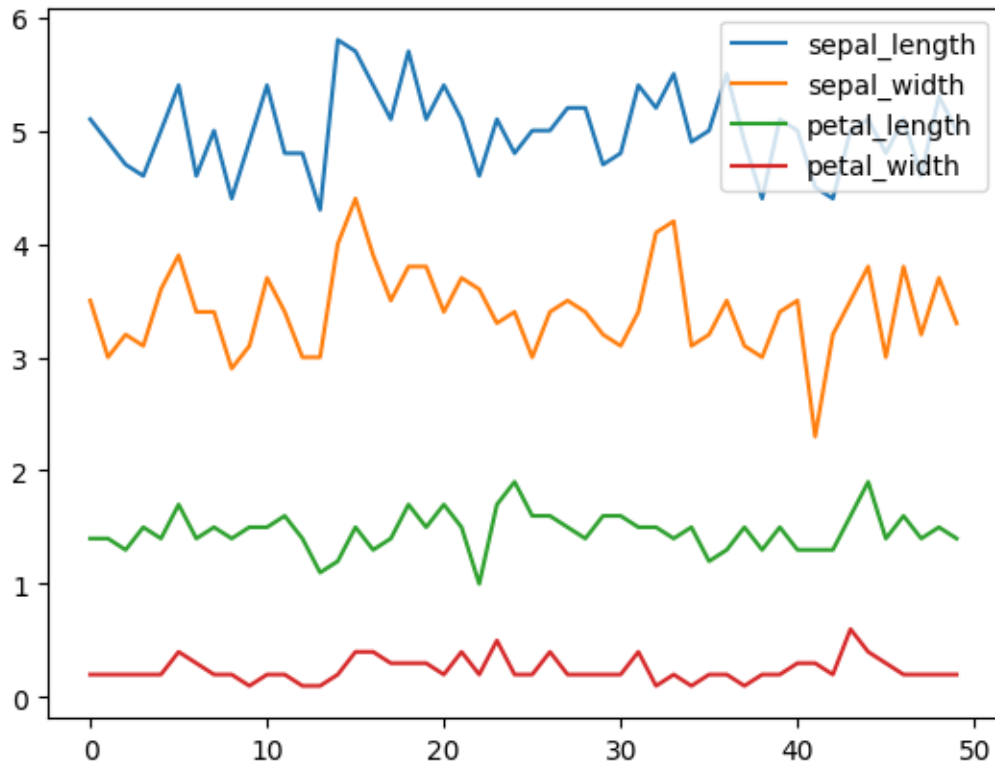
```


19	5.1	3.8	1.5	0.3	setosa
20	5.4	3.4	1.7	0.2	setosa
21	5.1	3.7	1.5	0.4	setosa
22	4.6	3.6	1.0	0.2	setosa
23	5.1	3.3	1.7	0.5	setosa
24	4.8	3.4	1.9	0.2	setosa
25	5.0	3.0	1.6	0.2	setosa
26	5.0	3.4	1.6	0.4	setosa
27	5.2	3.5	1.5	0.2	setosa
28	5.2	3.4	1.4	0.2	setosa
29	4.7	3.2	1.6	0.2	setosa
30	4.8	3.1	1.6	0.2	setosa
31	5.4	3.4	1.5	0.4	setosa
32	5.2	4.1	1.5	0.1	setosa
33	5.5	4.2	1.4	0.2	setosa
34	4.9	3.1	1.5	0.1	setosa
35	5.0	3.2	1.2	0.2	setosa
36	5.5	3.5	1.3	0.2	setosa
37	4.9	3.1	1.5	0.1	setosa
38	4.4	3.0	1.3	0.2	setosa
39	5.1	3.4	1.5	0.2	setosa
40	5.0	3.5	1.3	0.3	setosa
41	4.5	2.3	1.3	0.3	setosa
42	4.4	3.2	1.3	0.2	setosa
43	5.0	3.5	1.6	0.6	setosa
44	5.1	3.8	1.9	0.4	setosa
45	4.8	3.0	1.4	0.3	setosa
46	5.1	3.8	1.6	0.2	setosa
47	4.6	3.2	1.4	0.2	setosa
48	5.3	3.7	1.5	0.2	setosa
49	5.0	3.3	1.4	0.2	setosa

0.17 L'Iris Data Set: filtering

```
[18]: dataframe[dataframe["species"] == "setosa"].plot()
```

```
[18]: <Axes: >
```



0.18 L'Iris Data Set: filtering su più colonne

```
[19]: dataframe[(dataframe["species"] == "setosa") & (dataframe["petal_width"] > 0.3)]
```

```
[19]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
5	5.4	3.9	1.7	0.4	setosa
15	5.7	4.4	1.5	0.4	setosa
16	5.4	3.9	1.3	0.4	setosa
21	5.1	3.7	1.5	0.4	setosa
23	5.1	3.3	1.7	0.5	setosa
26	5.0	3.4	1.6	0.4	setosa
31	5.4	3.4	1.5	0.4	setosa
43	5.0	3.5	1.6	0.6	setosa
44	5.1	3.8	1.9	0.4	setosa

0.19 Plotting filtered DataFrame

```
[20]: setosa = dataframe[dataframe['species']=='setosa']
virginica = dataframe[dataframe['species']=='virginica']
versicolor = dataframe[dataframe['species']=='versicolor']
```

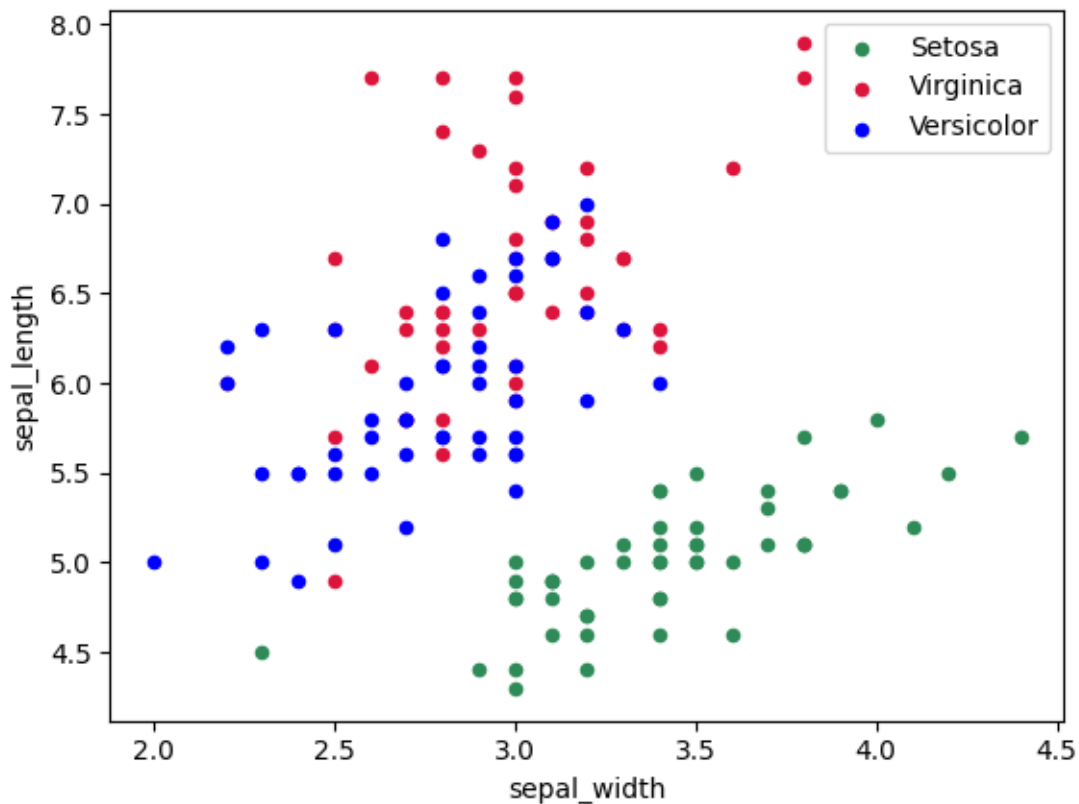
```

fig, ax = plt.subplots()

setosa.plot.scatter(x='sepal_width', y='sepal_length',
    ↪c='seagreen',label='Setosa',ax=ax )
virginica.plot.scatter(x='sepal_width',
    ↪y='sepal_length',c='crimson',label='Virginica', ax=ax)
versicolor.plot.scatter(x='sepal_width',
    ↪y='sepal_length',c='blue',label='Versicolor', ax=ax)

plt.legend()
plt.show()

```



0.20 Plotting DataFrames

```

[21]: import warnings
warnings.filterwarnings('ignore')
setosa = dataframe[dataframe['species']=='setosa']
virginica = dataframe[dataframe['species']=='virginica']
versicolor = dataframe[dataframe['species']=='versicolor']

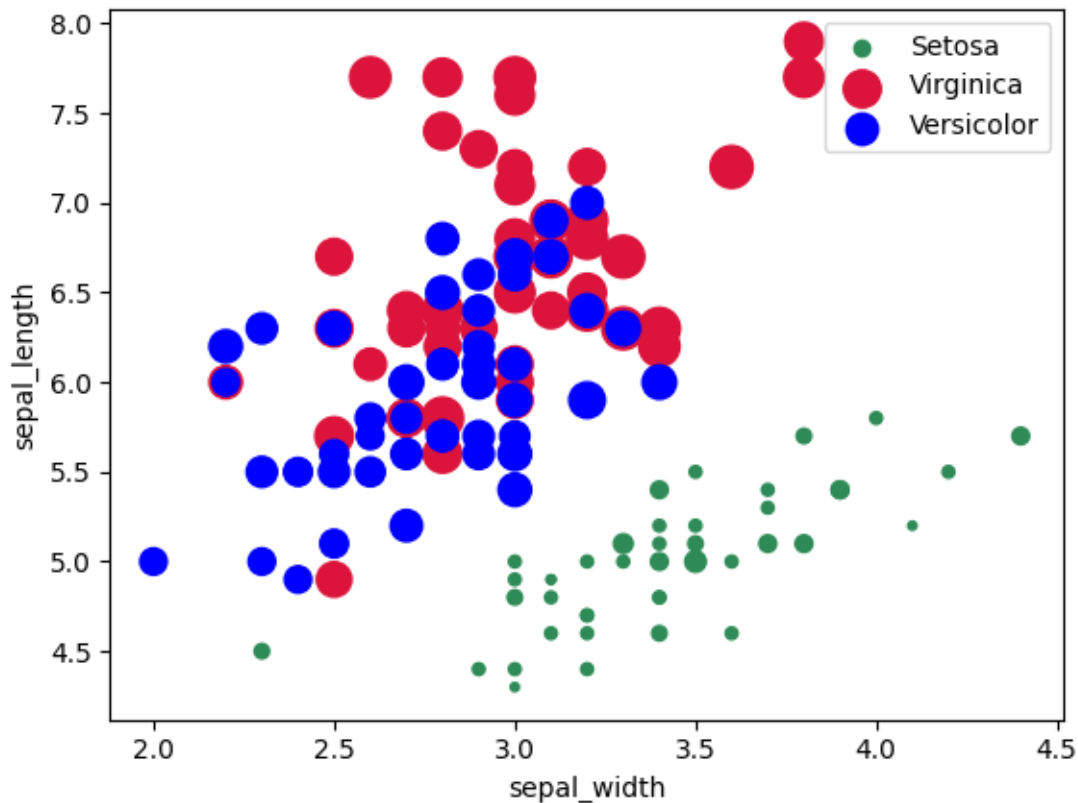
```

```

setosa.petal_width=setosa.petal_width*100
virginica.petal_width=virginica.petal_width*100
versicolor.petal_width=versicolor.petal_width*100

fig, ax = plt.subplots()
setosa.plot.scatter(x='sepal_width', y='sepal_length',
    ↪c='seagreen',s='petal_width', label='Setosa', ax=ax, )
virginica.plot.scatter(x='sepal_width',
    ↪y='sepal_length',c='crimson',s='petal_width', label='Virginica', ax=ax)
versicolor.plot.scatter(x='sepal_width',
    ↪y='sepal_length',c='blue',s='petal_width',label='Versicolor', ax=ax)
plt.legend()
plt.show()

```



0.21 Raggruppare: l'Iris DataSet

- immaginiamo di voler calcolare la dimensione media dei petali per ogni specie
- lo posso fare con gli strumenti visti fino a questo momento?

```

[22]: spw=dataframe[dataframe["species"] == "setosa"]["petal_width"].mean()
      vipw=dataframe[dataframe["species"] == "virginica"]["petal_width"].mean()

```

```
vepw=dataframe[dataframe["species"] == "versicolor"]["petal_width"].mean()
print(spw)
print(vipw)
print(vepw)
```

```
0.244
2.0260000000000002
1.3259999999999998
```

- ma il codice precedente è adatto quando si vuole calcolare la media **solo per poche specie**
- se i gruppi sono molti è necessario ripetere il codice precedente per ogni gruppo
- in questo caso è più opportuno usare la funzione **group_by**

0.22 Pandas: Group_By

- un modo per raggruppare le righe del *DataFrame* in *buckets*
- possibilità di
 - aggregare i valori in un gruppo
 - iterare sulle righe di un gruppo

```
[23]: import pandas as pd
food_data = {
    "Item": ["Banana", "Cucumber", "Orange", "Tomato", "Watermelon"],
    "Type": ["Fruit", "Vegetable", "Fruit", "Vegetable", "Fruit"],
    "Price": [0.99, 1.25, 0.25, 0.33, 3.00]
}
supermarket = pd.DataFrame(data = food_data)
supermarket
```

```
[23]:
```

	Item	Type	Price
0	Banana	Fruit	0.99
1	Cucumber	Vegetable	1.25
2	Orange	Fruit	0.25
3	Tomato	Vegetable	0.33
4	Watermelon	Fruit	3.00

0.23 Pandas: la funzione Group_By

- individuiamo i due gruppi *vegetables* e *fruits* nel *DataFrame*

```
[24]: groups_type = supermarket.groupby("Type")
groups_type
```

```
[24]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x000002426218B590>
```

- restituisce un oggetto di tipo **DataFrameGroupBy**
- implementato, a livello sottostante, con strutture **Python**

0.24 La funzione Group_By: under the wood

- come è implementato un oggetto di tipo **DataFrameGroupBy**?

```
[25]: groups_type.groups
```

```
[25]: {'Fruit': [0, 2, 4], 'Vegetable': [1, 3]}
```

```
[26]: groups_type.indices
```

```
[26]: {'Fruit': array([0, 2, 4], dtype=int64),  
      'Vegetable': array([1, 3], dtype=int64)}
```

- un **dizionario Python** che riporta per ogni gruppo
 - la chiave del gruppo, che corrisponde al valore dell'attributo rispetto a cui si sono formati i gruppi
 - un array con gli indici nel **DataFrame** di tutti gli appartenenti al gruppo
- consente a **Pandas** di ricostruire un **DataFrame** per ogni gruppo individuato

0.25 La funzione Group_By: under the wood

```
[27]: print (type(groups_type.get_group("Fruit")))  
      groups_type.get_group("Fruit")
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
[27]:
```

	Item	Type	Price
0	Banana	Fruit	0.99
2	Orange	Fruit	0.25
4	Watermelon	Fruit	3.00

- utilizzando l'oggetto **DataFrameGroupBy**, Pandas è in grado di costruire un **DataFrame** per ogni gruppo
- il metodo restituisce un **DataFrame** con tutte le righe degli elementi appartenenti al gruppo selezionato

0.26 La funzione Group_By: l'oggetto SeriesGroupBy

- è anche possibile selezionare una singola colonna riferendo l'oggetto **DataFrameGroupBY**
- **Pandas** costruisce una sequenza di Series, che corrispondono alla colonna corrispondente in ogni gruppo
- l'oggetto corrispondente è di tipo **SeriesGroupBy**
- utile quando si vuole applicare la funzione di aggregazione a una singola colonna

```
[28]: groups_type["Price"]  
      print(groups_type["Price"])
```

```
<pandas.core.groupby.generic.SeriesGroupBy object at 0x0000024261F687A0>
```

```
[29]: for keys, gp in groups_type['Price']:
      print(keys)
      print(gp)
```

```
Fruit
0    0.99
2    0.25
4    3.00
Name: Price, dtype: float64
Vegetable
1    1.25
3    0.33
Name: Price, dtype: float64
```

0.27 La funzione Group_By: l'oggetto SeriesGroupBy

```
[30]: sindexed=supermarket.set_index('Item')
      sindexed
```

```
[30]:
```

	Type	Price
Item		
Banana	Fruit	0.99
Cucumber	Vegetable	1.25
Orange	Fruit	0.25
Tomato	Vegetable	0.33
Watermelon	Fruit	3.00

```
[31]: groupsindexed=sindexed.groupby('Type')
      for keys, gp in groupsindexed['Price']:
          print(keys)
          print(gp)
```

```
Fruit
Item
Banana    0.99
Orange    0.25
Watermelon 3.00
Name: Price, dtype: float64
Vegetable
Item
Cucumber  1.25
Tomato    0.33
Name: Price, dtype: float64
```

0.28 La funzione Group_By: aggregare oggetti di tipo SeriesGroupBy

- possibile aggregare valori in singole colonne di ogni gruppo dell'oggetto **DataFrameGroupBy**

```
[32]: groups_type["Price"].mean()
```

```
[32]: Type
      Fruit      1.413333
      Vegetable  0.790000
      Name: Price, dtype: float64
```

0.29 La funzione Group_By: Iris DataSet

```
[33]: Inspecies= dataframe.groupby("species")
      species["petal_width"].mean()
```

```
[33]: species
      setosa      0.244
      versicolor  1.326
      virginica   2.026
      Name: petal_width, dtype: float64
```

0.30 La funzione Group_By: Iris DataSet

```
[34]: species.get_group('setosa').head()
```

```
[34]:   sepal_length  sepal_width  petal_length  petal_width  species
0           5.1           3.5           1.4           0.2   setosa
1           4.9           3.0           1.4           0.2   setosa
2           4.7           3.2           1.3           0.2   setosa
3           4.6           3.1           1.5           0.2   setosa
4           5.0           3.6           1.4           0.2   setosa
```

0.31 La funzione Group_By: IrisDataSet, aggregare su più campi

```
[35]: groups_iris = dataframe.groupby(["species", "petal_width"])
      groups_iris.indices
```

```
[35]: {('setosa', 0.1): array([ 9, 12, 13, 32, 34, 37], dtype=int64),
      ('setosa',
       0.2): array([ 0,  1,  2,  3,  4,  7,  8, 10, 11, 14, 20, 22, 24, 25, 27, 28,
       29,
                  30, 33, 35, 36, 38, 39, 42, 46, 47, 48, 49], dtype=int64),
      ('setosa', 0.3): array([ 6, 17, 18, 19, 40, 41, 45], dtype=int64),
      ('setosa', 0.4): array([ 5, 15, 16, 21, 26, 31, 44], dtype=int64),
      ('setosa', 0.5): array([23], dtype=int64),
      ('setosa', 0.6): array([43], dtype=int64),
      ('versicolor', 1.0): array([57, 60, 62, 67, 79, 81, 93], dtype=int64),
      ('versicolor', 1.1): array([69, 80, 98], dtype=int64),
      ('versicolor', 1.2): array([73, 82, 90, 92, 95], dtype=int64),
```



```

('versicolor',
 1.3): array([53, 55, 58, 64, 71, 74, 87, 88, 89, 94, 96, 97, 99],
dtype=int64),
('versicolor', 1.4): array([50, 59, 63, 65, 75, 76, 91], dtype=int64),
('versicolor',
 1.5): array([51, 52, 54, 61, 66, 68, 72, 78, 84, 86], dtype=int64),
('versicolor', 1.6): array([56, 83, 85], dtype=int64),
('versicolor', 1.7): array([77], dtype=int64),
('versicolor', 1.8): array([70], dtype=int64),
('virginica', 1.4): array([134], dtype=int64),
('virginica', 1.5): array([119, 133], dtype=int64),
('virginica', 1.6): array([129], dtype=int64),
('virginica', 1.7): array([106], dtype=int64),
('virginica',
 1.8): array([103, 107, 108, 116, 123, 125, 126, 127, 137, 138, 149],
dtype=int64),
('virginica', 1.9): array([101, 111, 130, 142, 146], dtype=int64),
('virginica', 2.0): array([110, 113, 121, 122, 131, 147], dtype=int64),
('virginica', 2.1): array([102, 105, 112, 124, 128, 139], dtype=int64),
('virginica', 2.2): array([104, 117, 132], dtype=int64),
('virginica',
 2.3): array([115, 118, 120, 135, 141, 143, 145, 148], dtype=int64),
('virginica', 2.4): array([114, 136, 140], dtype=int64),
('virginica', 2.5): array([100, 109, 144], dtype=int64)}

```

0.32 Criptocurrencies

- molta attenzione da parte di
 - investitori
 - imprenditori
 - regolatori
 - pubblico in generale
- molti argomenti di dibattito
 - solo un asset o una vera moneta?
 - Initial Coin Offering (ICO) per il finanziamento di start-up innovative
 - monete emesse da banche centrali, ma che usano la tecnologia alla base delle criptomonete, cioè la blockchain
 - il mercato delle cryptocurrencies si rivelerà solo una “bolla”?
 - * regolazione o divieto?
- grande interesse alle analisi di mercato
 - big data analysis
 - machine learning
- in questa lezione presenteremo un insieme di semplici analisi del dataset *crypto-markets.csv*

0.33 Il Data Set *crypto-markets.csv*: analisi delle caratteristiche

```
[36]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime

import warnings
warnings.filterwarnings('ignore')

import matplotlib.dates as mdates

# Numero di criptocurrencies che analizzeremo in diverse analisi
x = 10
```

0.34 Il Data Set *crypto-markets.csv*: analisi delle caratteristiche

```
[38]: df_crypto = pd.read_csv('DataSets/crypto-markets.csv', parse_dates = ["date"])
df_crypto.info()
df_crypto.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 679183 entries, 0 to 679182
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  -
0   slug             679183 non-null object
1   symbol           679183 non-null object
2   name             679183 non-null object
3   date             679183 non-null datetime64[ns]
4   ranknow          679183 non-null int64
5   open            679183 non-null float64
6   high            679183 non-null float64
7   low             679183 non-null float64
8   close           679183 non-null float64
9   volume          679183 non-null int64
10  market          679183 non-null float64
11  close_ratio     665926 non-null object
12  spread          679183 non-null float64
dtypes: datetime64[ns](1), float64(6), int64(2), object(4)
memory usage: 67.4+ MB
```

```
[38]:      slug symbol    name    date  ranknow   open   high   low  \
0  bitcoin   BTC  Bitcoin 2013-04-28         1  135.30  135.98  132.10
1  bitcoin   BTC  Bitcoin 2013-04-29         1  134.44  147.49  134.00
2  bitcoin   BTC  Bitcoin 2013-04-30         1  144.00  146.93  134.05
```

3	bitcoin	BTC	Bitcoin	2013-05-01	1	139.00	139.89	107.72
4	bitcoin	BTC	Bitcoin	2013-05-02	1	116.38	125.60	92.28

	close	volume	market	close_ratio	spread
0	134.21	0	1.500520e+09	0.5438	3.88
1	144.54	0	1.491160e+09	0.7813	13.49
2	139.00	0	1.597780e+09	0.3843	12.88
3	116.99	0	1.542820e+09	0.2882	32.17
4	105.21	0	1.292190e+09	0.3881	33.32

0.35 Criptocurrencies

- Una riga per ogni giorno in un periodo compreso tra due date
 - *slug* : nome comune della cryptocurrency
 - *symbol* : simbolo della cryptocurrency
 - *name* : nome ufficiale della cryptocurrency
 - *date*: data dell'osservazione
 - *ranknow* : rank della cryptocurrency in quella data
 - *open* : prezzo di apertura in quella data
 - *high* : prezzo più alto in quella data
 - *low* : prezzo più basso in quella data
 - *close* : prezzo di chiusura in quella data
 - *volume*: volume delle transazioni in quella data
 - *market* : capitalizzazione di mercato in USD
 - *close_ratio*: (close-low)/(high-low)
 - *spread* : differenza tra il prezzo più alto e quello più basso in quella data
- periodo coperto: *2013-2018*
- dimensione: circa *62 Mega*

0.36 Il Data Set *crypto-markets.csv*: analisi delle caratteristiche

- da notare che il tipo della colonna date è cambiato in *datetime64[ns]*
- anche la notazione usata per visualizzare la data è cambiata

```
[39]: df_crypto.tail()
```

```
[39]:
```

	slug	symbol	name	date	ranknow	open	high	\
679178	bitcoin-atom	BCA	Bitcoin Atom	2018-02-01	1515	45.95	55.17	
679179	bitcoin-atom	BCA	Bitcoin Atom	2018-02-02	1515	42.90	42.90	
679180	bitcoin-atom	BCA	Bitcoin Atom	2018-02-03	1515	35.99	36.71	
679181	bitcoin-atom	BCA	Bitcoin Atom	2018-02-04	1515	30.75	34.90	
679182	bitcoin-atom	BCA	Bitcoin Atom	2018-02-05	1515	25.52	27.42	

	low	close	volume	market	close_ratio	spread
679178	41.70	42.95	38029	0.0	0.0928	13.47
679179	29.18	35.91	48024	0.0	0.4905	13.72
679180	30.10	30.74	27136	0.0	0.0968	6.61
679181	23.93	23.93	26370	0.0	0.0	10.97

679182 19.25 19.68 17804 0.0 0.0526 8.17

0.37 Quotazioni rilevate nella data più recente

```
[40]: # trovare la data più recente
mostrecentdate=max(df_crypto['date'])
print(mostrecentdate)

# Costruire un nuovo DataFrame contenente, per ogni crypto, la quotazione più
↪ recente

print("I dati più recenti sulle crypto")
latest_df = df_crypto[df_crypto['date'] == mostrecentdate]
latest_df.head()
```

2018-02-05 00:00:00

I dati più recenti sulle crypto

```
[40]:
```

	slug	symbol	name	date	ranknow	open	\
1744	bitcoin	BTC	Bitcoin	2018-02-05	1	8270.540000	
2658	ethereum	ETH	Ethereum	2018-02-05	2	834.960000	
4305	ripple	XRP	Ripple	2018-02-05	3	0.825943	
4503	bitcoin-cash	BCH	Bitcoin Cash	2018-02-05	4	1159.890000	
4631	cardano	ADA	Cardano	2018-02-05	5	0.385044	

	high	low	close	volume	market	\
1744	8364.840000	6756.680000	6955.270000	9285290000	1.393250e+11	
2658	856.650000	644.600000	697.950000	4269530000	8.133740e+10	
4305	0.833866	0.675827	0.691799	1286180000	3.221940e+10	
4503	1172.700000	839.950000	887.410000	568117000	1.965980e+10	
4631	0.399523	0.319646	0.327371	658548000	9.983060e+09	

	close_ratio	spread
1744	0.1235	1608.16
2658	0.2516	212.05
4305	0.1011	0.16
4503	0.1426	332.75
4631	0.0967	0.08

0.38 Selezionare la data della prima rilevazione di ogni cryptocurrency

```
[41]: df_grouped=df_crypto.groupby('name')
df_grouped['date'].min()
```

```
[41]: name
0x      2017-08-16
```

```

10M Token      2017-10-24
2GIVE          2016-05-16
300 Token      2017-07-25
42-coin        2014-01-14
...
netBit         2016-11-14
onG.social     2017-12-01
ugChain        2018-01-13
vSlice         2016-12-19
vTorrent       2014-12-25
Name: date, Length: 1511, dtype: datetime64[ns]

```

0.39 Selezionare la data della prima rilevazione di ogni cryptocurrency

```
[42]: start_df = pd.DataFrame({'start_date' : df_grouped['date'].min()})
      start_df
```

```
[42]:          start_date
name
0x          2017-08-16
10M Token   2017-10-24
2GIVE       2016-05-16
300 Token   2017-07-25
42-coin     2014-01-14
...
netBit       2016-11-14
onG.social   2017-12-01
ugChain      2018-01-13
vSlice       2016-12-19
vTorrent     2014-12-25

[1511 rows x 1 columns]
```

0.40 Selezionare le criptovalute che sono state entrate per prime nel mercato

```
[43]: print("Oldest Cryptocurrencies")

      # x è impostato a 10 inizialmente

      start_df.sort_values(['start_date']).head(x)
```

Oldest Cryptocurrencies

```
[43]:          start_date
name
Peercoin     2013-04-28
```

Litecoin	2013-04-28
Bitcoin	2013-04-28
Namecoin	2013-04-28
Novacoin	2013-04-28
Terracoin	2013-04-28
Mincoin	2013-05-03
Freicoin	2013-05-03
Feathercoin	2013-05-03
Ixcoin	2013-05-08

0.41 Selezionare le ultime cryptocurrencies entrate sul mercato (nel 2018)

```
[44]: # List the newest ones
print("Newest Cryptocurrencies")

start_df.sort_values(['start_date']).tail(x)
```

Newest Cryptocurrencies

```
[44]:          start_date
name
Pareto Network  2018-02-02
STK             2018-02-02
Polymath Network 2018-02-02
Huobi Token     2018-02-03
SwissBorg       2018-02-03
adbank          2018-02-03
DMarket         2018-02-03
Zilla           2018-02-03
Iungo           2018-02-04
Jibrel Network  2018-02-05
```

0.42 Data wrangling

```
[45]: df_crypto['market_billion'] = df_crypto['market'] / 1000000000
df_crypto['volume_million'] = df_crypto['volume'] / 1000000000

print(df_crypto)
```

	slug	symbol	name	date	ranknow	open	high	\
0	bitcoin	BTC	Bitcoin	2013-04-28	1	135.30	135.98	
1	bitcoin	BTC	Bitcoin	2013-04-29	1	134.44	147.49	
2	bitcoin	BTC	Bitcoin	2013-04-30	1	144.00	146.93	
3	bitcoin	BTC	Bitcoin	2013-05-01	1	139.00	139.89	
4	bitcoin	BTC	Bitcoin	2013-05-02	1	116.38	125.60	
...	...	...	...	...	...	...	...	...
679178	bitcoin-atom	BCA	Bitcoin Atom	2018-02-01	1515	45.95	55.17	

679179	bitcoin-atom	BCA	Bitcoin Atom	2018-02-02	1515	42.90	42.90
679180	bitcoin-atom	BCA	Bitcoin Atom	2018-02-03	1515	35.99	36.71
679181	bitcoin-atom	BCA	Bitcoin Atom	2018-02-04	1515	30.75	34.90
679182	bitcoin-atom	BCA	Bitcoin Atom	2018-02-05	1515	25.52	27.42

	low	close	volume	market	close_ratio	spread	\
0	132.10	134.21	0	1.500520e+09	0.5438	3.88	
1	134.00	144.54	0	1.491160e+09	0.7813	13.49	
2	134.05	139.00	0	1.597780e+09	0.3843	12.88	
3	107.72	116.99	0	1.542820e+09	0.2882	32.17	
4	92.28	105.21	0	1.292190e+09	0.3881	33.32	
...	...	...	...	...	...	...	
679178	41.70	42.95	38029	0.000000e+00	0.0928	13.47	
679179	29.18	35.91	48024	0.000000e+00	0.4905	13.72	
679180	30.10	30.74	27136	0.000000e+00	0.0968	6.61	
679181	23.93	23.93	26370	0.000000e+00	0.0	10.97	
679182	19.25	19.68	17804	0.000000e+00	0.0526	8.17	

	market_billion	volume_million
0	1.50052	0.000000
1	1.49116	0.000000
2	1.59778	0.000000
3	1.54282	0.000000
4	1.29219	0.000000
...	...	...
679178	0.00000	0.000038
679179	0.00000	0.000048
679180	0.00000	0.000027
679181	0.00000	0.000026
679182	0.00000	0.000018

[679183 rows x 15 columns]

0.43 Individuare le top cryptocurrencies in base al ranking rilevato nella data più recente

- top 10: rank =1 topmost, selezioniamo per rank <=10
- valutazione relativa all'ultima rilevazione di mercato effettuata
- **latest_df**: è il **DataFrame** costruito nelle slide precedenti, che contiene le rilevazioni effettuate nell'ultima data del DataSet

```
[46]: high_rank=latest_df[latest_df['ranknow'] <= x]
      high_rank
```

```
[46]: slug symbol name date ranknow open \
1744 bitcoin BTC Bitcoin 2018-02-05 1 8270.540000
2658 ethereum ETH Ethereum 2018-02-05 2 834.960000
4305 ripple XRP Ripple 2018-02-05 3 0.825943
```

4503	bitcoin-cash	BCH	Bitcoin Cash	2018-02-05	4	1159.890000
4631	cardano	ADA	Cardano	2018-02-05	5	0.385044
6376	litecoin	LTC	Litecoin	2018-02-05	6	148.730000
6891	neo	NEO	NEO	2018-02-05	7	107.580000
8172	stellar	XLM	Stellar	2018-02-05	8	0.383191
8392	eos	EOS	EOS	2018-02-05	9	8.680000
9434	nem	XEM	NEM	2018-02-05	10	0.548980

	high	low	close	volume	market \
1744	8364.840000	6756.680000	6955.270000	9285290000	1.393250e+11
2658	856.650000	644.600000	697.950000	4269530000	8.133740e+10
4305	0.833866	0.675827	0.691799	1286180000	3.221940e+10
4503	1172.700000	839.950000	887.410000	568117000	1.965980e+10
4631	0.399523	0.319646	0.327371	658548000	9.983060e+09
6376	153.520000	118.150000	125.330000	844678000	8.190580e+09
6891	108.970000	75.750000	82.370000	312096000	6.992380e+09
8172	0.425340	0.312975	0.331192	339507000	7.064590e+09
8392	8.850000	6.940000	7.060000	640159000	5.618380e+09
9434	0.559763	0.440340	0.449672	28590400	4.940820e+09

	close_ratio	spread
1744	0.1235	1608.16
2658	0.2516	212.05
4305	0.1011	0.16
4503	0.1426	332.75
4631	0.0967	0.08
6376	0.203	35.37
6891	0.1993	33.22
8172	0.1621	0.11
8392	0.0628	1.91
9434	0.0781	0.12

0.44 Le top 10 cryptocurrencies in base al ranking rilevato nella data più recente

```
[47]: high_rank.sort_values(by=['market'], ascending=[False])
      high_rank[['name', 'market']].head(x)
```

```
[47]:
```

	name	market
1744	Bitcoin	1.393250e+11
2658	Ethereum	8.133740e+10
4305	Ripple	3.221940e+10
4503	Bitcoin Cash	1.965980e+10
4631	Cardano	9.983060e+09
6376	Litecoin	8.190580e+09
6891	NEO	6.992380e+09
8172	Stellar	7.064590e+09
8392	EOS	5.618380e+09

9434

NEM 4.940820e+09

0.45 Le top 10 cryptocurrencies per capitalizzazione

- creare un plot che riporti le prime 10 cryptocurrencies per market capitalization
- in riferimento alla data dell'ultima rilevazione, per quella criptomoneta
- usiamo la funzione **last**, invece del **DataFrame latestDf**

```
[48]: mv = df_crypto.groupby('name')['market_billion'].last()  
mv
```

```
[48]: name  
0x                0.579817  
10M Token         0.000000  
2GIVE             0.004693  
300 Token         0.000256  
42-coin           0.002731  
...  
netBit            0.000000  
onG.social        0.008939  
ugChain           0.000000  
vSlice            0.004289  
vTorrent          0.004629  
Name: market_billion, Length: 1511, dtype: float64
```

0.46 Le top 10 cryptocurrencies per capitalizzazione

- utilizziamo la funzione **last**
- verifichiamo che effettivamente tale funzione restituisce l'ultima capitalizzazione di mercato

```
[49]: df_crypto.loc[df_crypto['slug']=="0x"].tail()
```

```
[49]:      slug symbol name      date  ranknow  open  high      low      close  \  
24868  0x    ZRX  0x 2018-02-01      40  1.86  1.93  1.440000  1.600000  
24869  0x    ZRX  0x 2018-02-02      40  1.61  1.61  1.050000  1.300000  
24870  0x    ZRX  0x 2018-02-03      40  1.30  1.50  1.150000  1.390000  
24871  0x    ZRX  0x 2018-02-04      40  1.39  1.39  1.050000  1.150000  
24872  0x    ZRX  0x 2018-02-05      40  1.15  1.16  0.735911  0.817119  
  
      volume      market  close_ratio  spread  market_billion  \  
24868  30422600  936047000.0      0.3265   0.49      0.936047  
24869  26332600  807909000.0      0.4464   0.56      0.807909  
24870  14554900  655192000.0      0.6857   0.35      0.655192  
24871  13450500  703996000.0      0.2941   0.34      0.703996  
24872  16094600  579817000.0      0.1915   0.42      0.579817  
  
      volume_million
```

24868	0.030423
24869	0.026333
24870	0.014555
24871	0.013451
24872	0.016095

0.47 Le top 10 cryptocurrencies per capitalizzazione

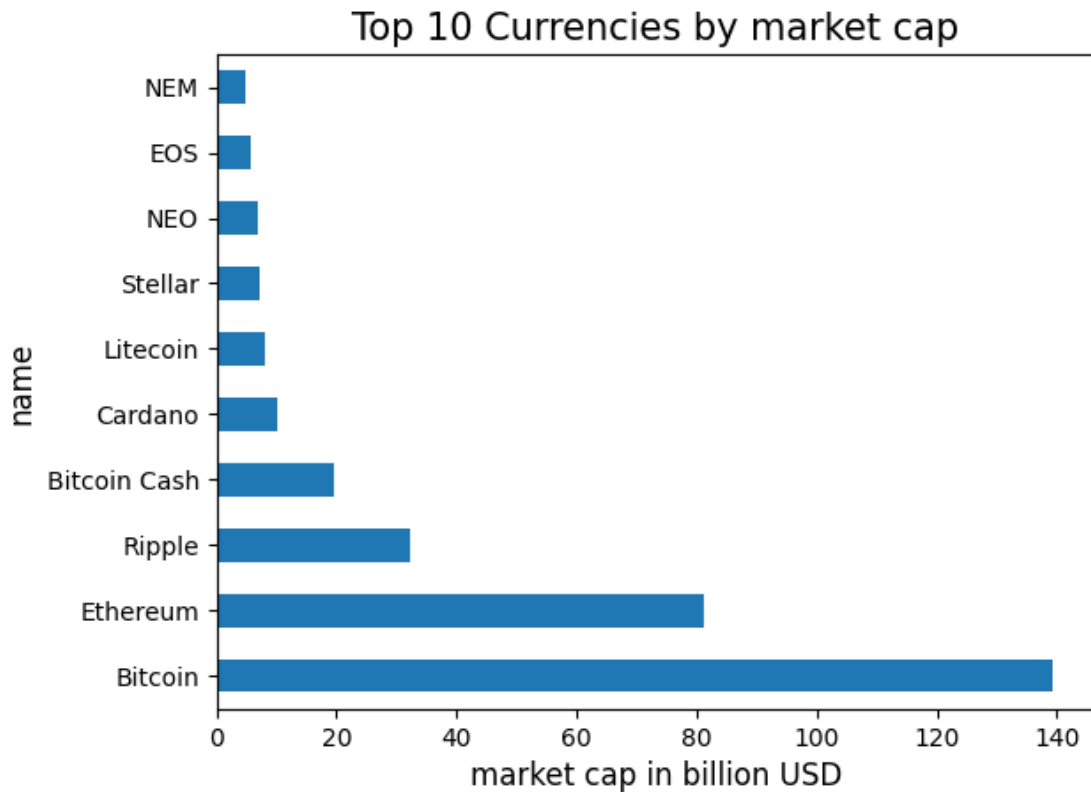
```
[50]: mv= mv.sort_values(ascending=False)
      mv.head(10)
```

```
[50]: name
      Bitcoin      139.32500
      Ethereum      81.33740
      Ripple       32.21940
      Bitcoin Cash  19.65980
      Cardano       9.98306
      Litecoin      8.19058
      Stellar       7.06459
      NEO           6.99238
      EOS           5.61838
      NEM           4.94082
      Name: market_billion, dtype: float64
```

0.48 Le top 10 cryptocurrencies per capitalizzazione

```
[51]: mv.head(10).plot(kind='barh')
      plt.xlabel("market cap in billion USD", size = 12)
      plt.ylabel("name", size = 12)
      plt.title("Top 10 Currencies by market cap", size = 15)
```

```
[51]: Text(0.5, 1.0, 'Top 10 Currencies by market cap')
```



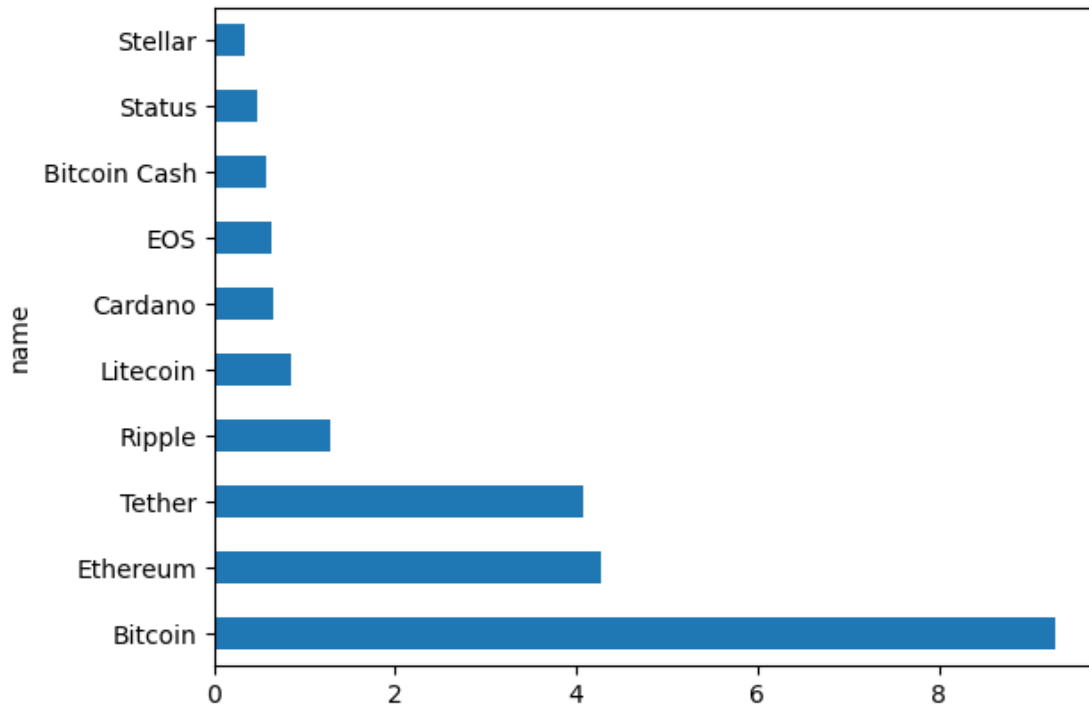
- `barh`: per ottenere istogrammi in orizzontale

0.49 Le prime 10 cryptocurrencies per volume

```
[52]: high_volume = df_crypto.groupby('name')['volume_million'].last()
      high_volume = high_volume.sort_values(ascending=False)
      high_volume.head(10)

      high_volume.head(10).plot(kind='barh')
```

```
[52]: <Axes: ylabel='name'>
```



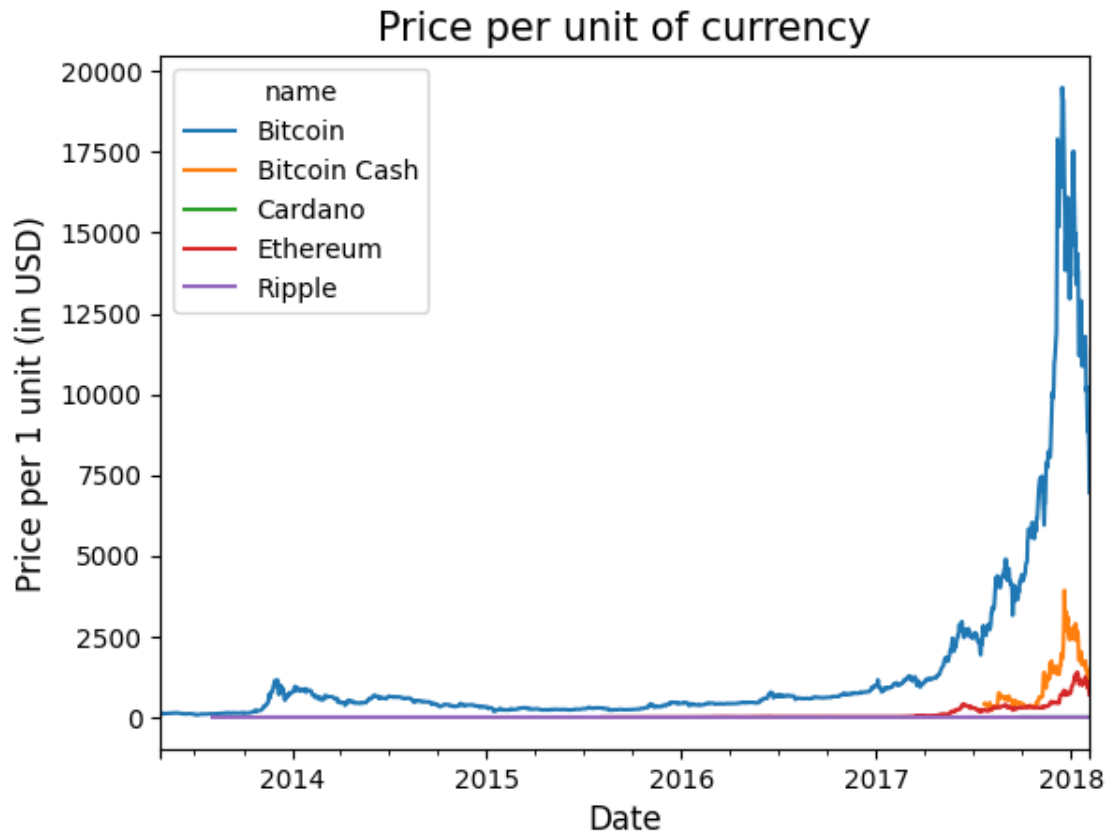
0.50 Ex 2: Andamento del prezzo delle top-5 cryptocurrencies

```
[53]: market = df_crypto.groupby(['name'])['market'].last()
top_5_currency_names =market.sort_values(ascending=False).head(5).index
print (top_5_currency_names)

data_top_5_currencies = df_crypto[df_crypto['name'].isin(top_5_currency_names)]
data=data_top_5_currencies.groupby(['date', 'name'])['close'].last()
data.unstack().plot()
plt.xlabel("Date", size = 12)
plt.ylabel("Price per 1 unit (in USD)", size = 12)
plt.title("Price per unit of currency", size = 15)
```

```
Index(['Bitcoin', 'Ethereum', 'Ripple', 'Bitcoin Cash', 'Cardano'],
      dtype='object', name='name')
```

```
[53]: Text(0.5, 1.0, 'Price per unit of currency')
```



0.51 Andamento del prezzo delle top 5 cryptocurrencies

- cosa accade se tolgo l'istruzione `unstack()`?

```
[54]: market = df_crypto.groupby(['name'])['market'].last()
top_5_currency_names = market.sort_values(ascending=False).head(5).index
print (top_5_currency_names)

data_top_5_currencies = df_crypto[df_crypto['name'].isin(top_5_currency_names)]

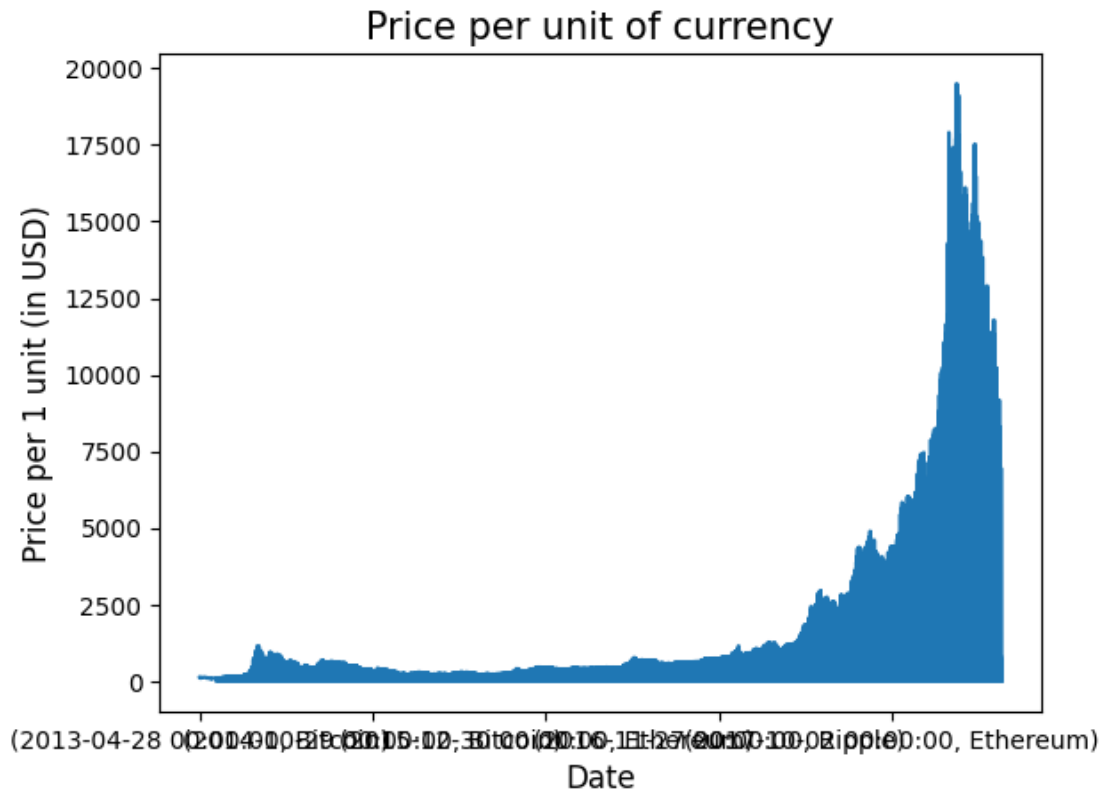
data=data_top_5_currencies.groupby(['date', 'name'])['close'].last()

data.plot()

plt.xlabel("Date", size = 12)
plt.ylabel("Price per 1 unit (in USD)", size = 12)
plt.title("Price per unit of currency", size = 15)
```

```
Index(['Bitcoin', 'Ethereum', 'Ripple', 'Bitcoin Cash', 'Cardano'],
      dtype='object', name='name')
```

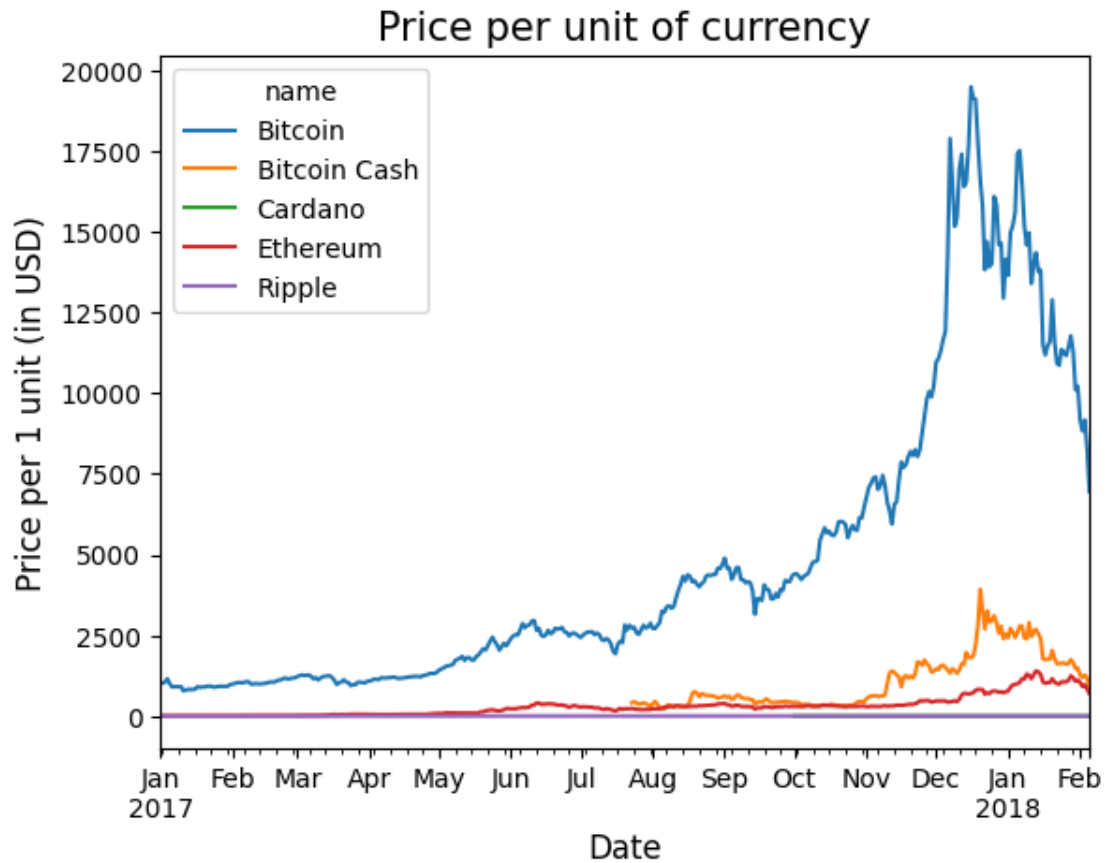
```
[54]: Text(0.5, 1.0, 'Price per unit of currency')
```



0.52 Andamento del prezzo nel 2017 top 5 cryptocurrencies

```
[55]: price2017 = data_top_5_currencies[data_top_5_currencies.date.dt.year >= 2017].
      ↳groupby(['date', 'name'])['close'].last()
price2017.unstack().plot();
plt.xlabel("Date", size = 12)
plt.ylabel("Price per 1 unit (in USD)", size = 12)
plt.title("Price per unit of currency", size = 15)
```

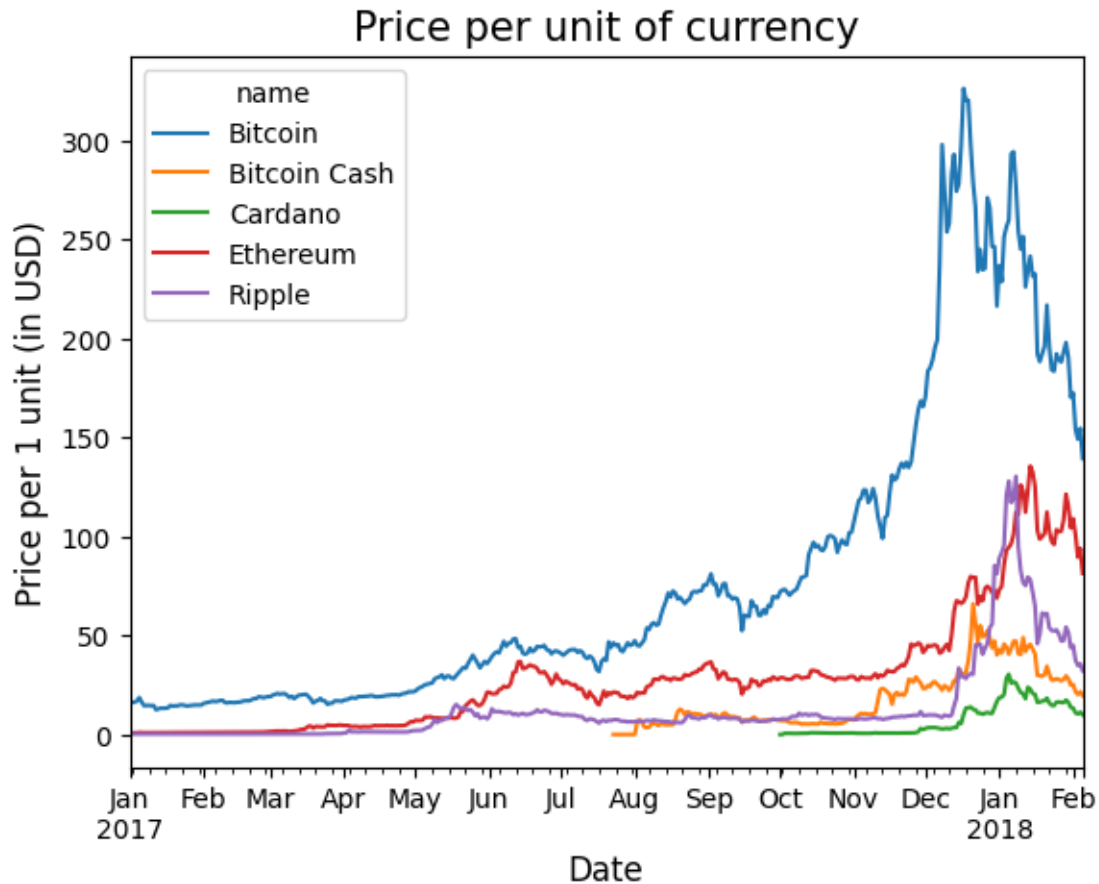
```
[55]: Text(0.5, 1.0, 'Price per unit of currency')
```



0.53 Capitalizzazione nel 2017 top 5 cryptocurrencies

```
[56]: exprice2017 = data_top_5_currencies[data_top_5_currencies.date.dt.year >= 2017].
      ↳groupby(['date', 'name'])['market_billion'].last()
price2017.unstack().plot();
plt.xlabel("Date", size = 12)
plt.ylabel("Price per 1 unit (in USD)", size = 12)
plt.title("Price per unit of currency", size = 15)
```

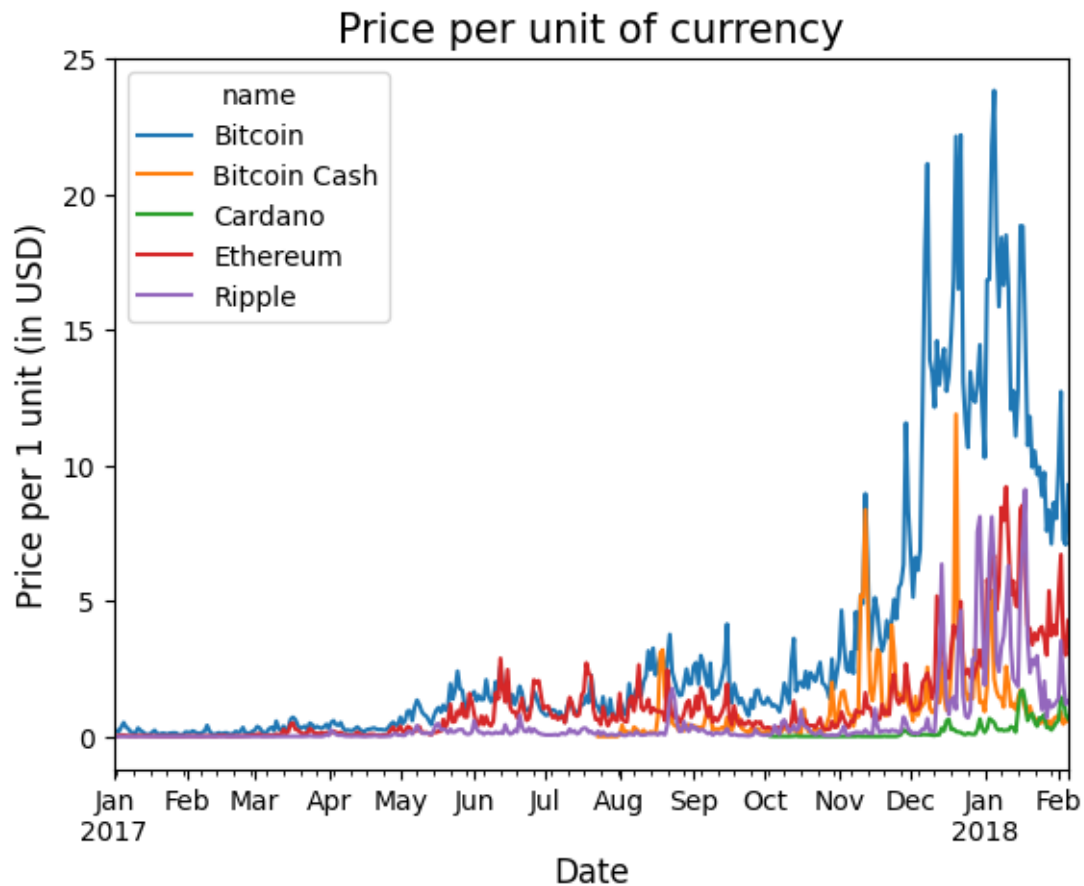
```
[56]: Text(0.5, 1.0, 'Price per unit of currency')
```



0.54 Volume dal 2017

```
[57]: price2017 = data_top_5_currencies[data_top_5_currencies.date.dt.year >= 2017].
      ↳groupby(['date', 'name'])['volume_million'].last()
price2017.unstack().plot();
plt.xlabel("Date", size = 12)
plt.ylabel("Price per 1 unit (in USD)", size = 12)
plt.title("Price per unit of currency", size = 15)
```

```
[57]: Text(0.5, 1.0, 'Price per unit of currency')
```

0.55 Approfondimenti: Seaborn, HeatMap

```
[58]: import matplotlib.pyplot as plt
import pandas as pd

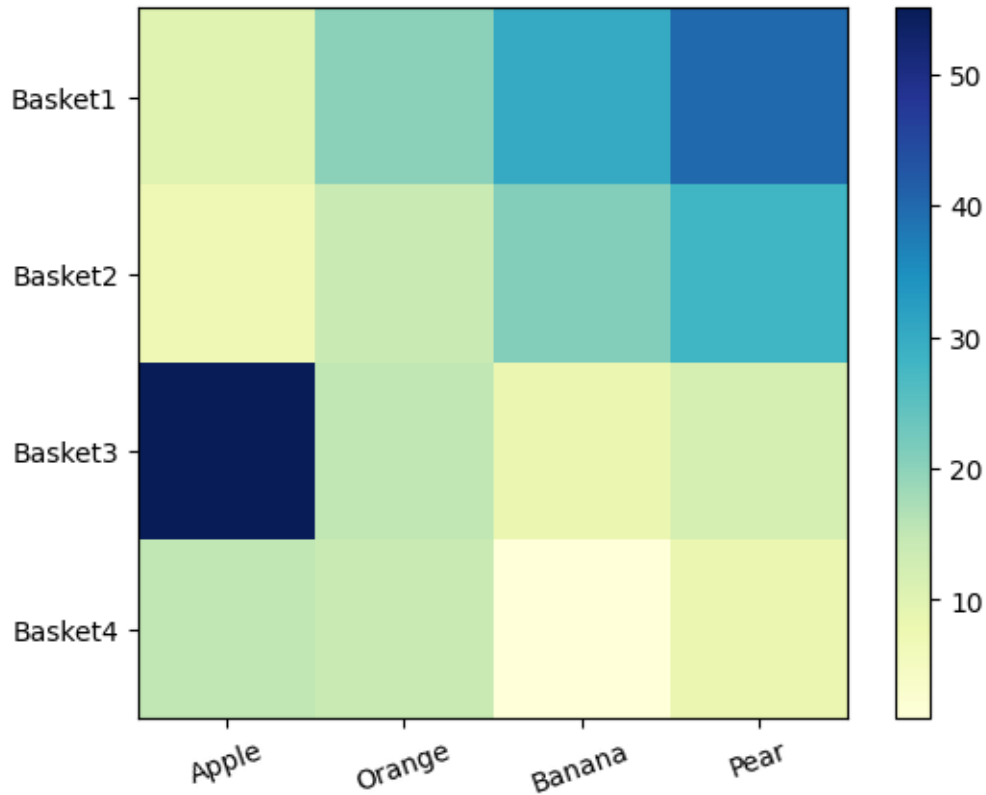
df = pd.DataFrame([[10, 20, 30, 40], [7, 14, 21, 28], [55, 15, 8, 12],
                  [15, 14, 1, 8]],
                  columns=['Apple', 'Orange', 'Banana', 'Pear'],
                  index=['Basket1', 'Basket2', 'Basket3', 'Basket4']
                  )

print(df)

plt.imshow(df, cmap="YlGnBu")
plt.colorbar()
plt.xticks(range(len(df)),df.columns, rotation=20)
plt.yticks(range(len(df)),df.index)
plt.show()
```

Apple Orange Banana Pear

Basket1	10	20	30	40
Basket2	7	14	21	28
Basket3	55	15	8	12
Basket4	15	14	1	8



0.56 Approfondimenti: HeatMap, correlazione

- una matrice che riporta le stesse variabili sull'asse x e sull'asse y
- ogni elemento della matrice riporta la correlazione tra le variabili corrispondenti
- correlazione varia tra -1 e 1
- valori vicini a 1 indicano che le variabili sono molto correlate
 - se incremento una variabile, anche l'altra incrementa
- valori vicini a -1 indicano l'inverso
 - se incremento una variabile, l'altra diminuisce
- le diagonali correlano una variabile con se stessa e quindi riportano sempre valori =1
 - correlazione massima
- più alta la correlazione, più scuro il valore

- plot simmetrico rispetto alla diagonale

```
[60]: import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

countries = ["china", "usa", "france", "russia",
             "japan", "india", "UK"]
GDP_trillion = [16, 23, 2, 1, 5, 3, 3]
Population = [1.4, 0.3, 0.068, 0.14, 0.12, 1.3, 0.067]
GDP_per_capita = [2.12, 3.45, 6.23, 7.89, 4.23, 5.34, 6.9]
Agricultural_land = [0.92, 1.2, 0.45, 0.73, 0.21, 0.34, 0.91]
Co2_emission = [0.40, 0.45, 0.34, 0.23, 0.97, 0.21, 0.74]

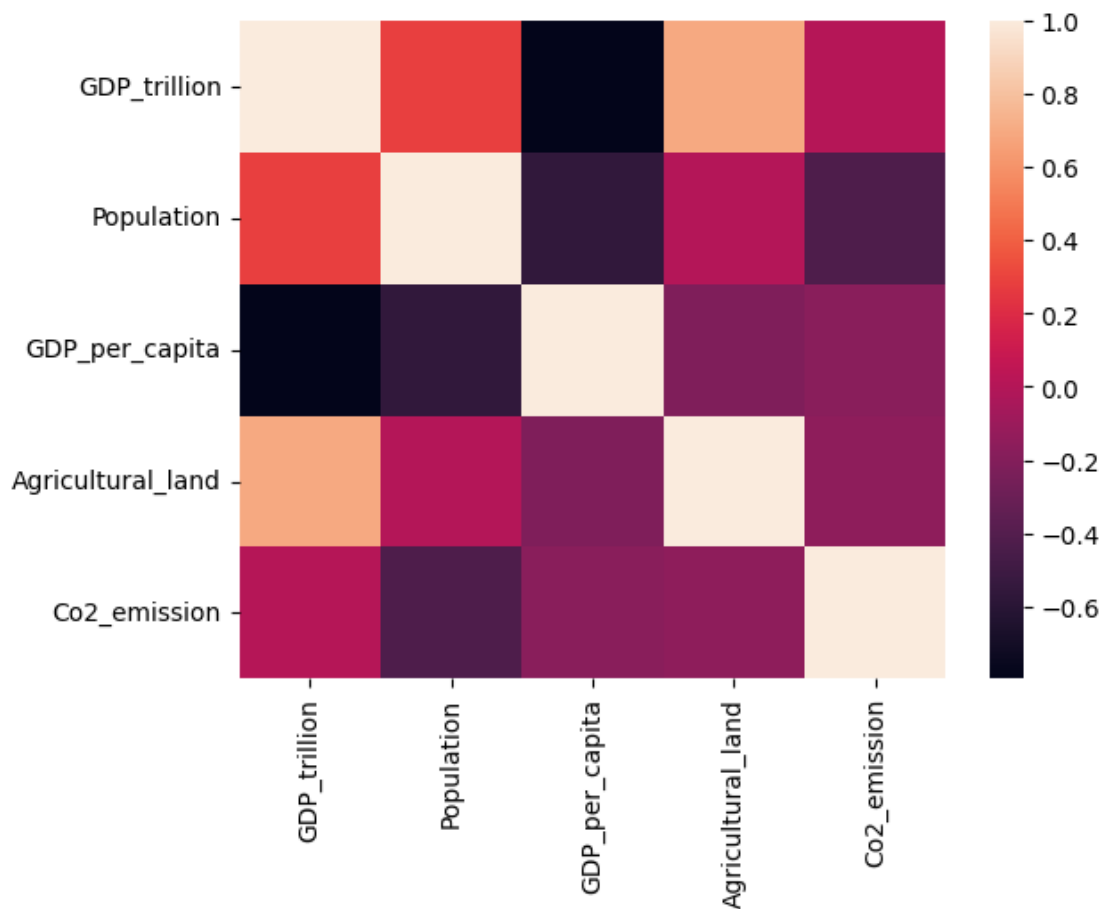
df=pd.DataFrame({
    'countries': countries,
    'GDP_trillion': GDP_trillion,
    'Population': Population,
    'GDP_per_capita': GDP_per_capita,
    'Agricultural_land': Agricultural_land,
    'Co2_emission': Co2_emission
})

df1=df.set_index('countries')

2df1

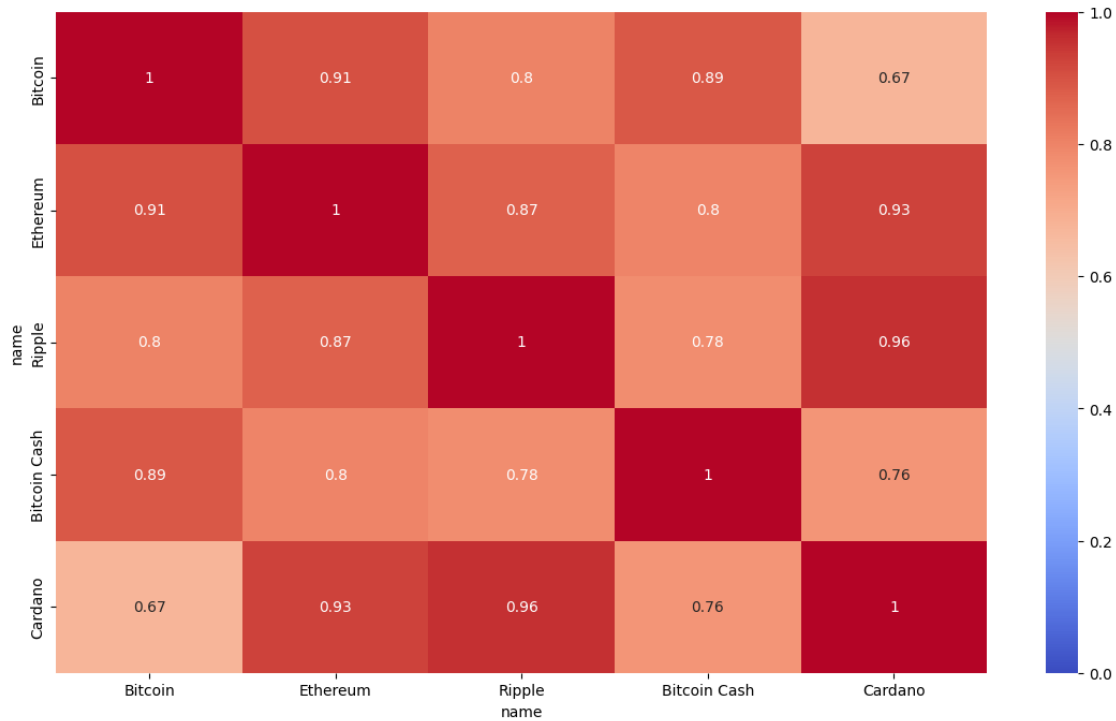
pear_corr=df1.corr(method='pearson')
sns.heatmap(pear_corr)
plt.show
```

```
[60]: <function matplotlib.pyplot.show(close=None, block=None)>
```



0.57 Ex 2: Correlazioni

```
[61]: wide_format = df_crypto.groupby(['date', 'name'])['close'].last().unstack()
plt.figure(figsize=(14,8))
sns.heatmap(wide_format[top_5_currency_names].corr(method='pearson'),vmin=0,
            ↪vmax=1, cmap='coolwarm', annot=True);
```



0.58 Assignment 5

- netflix.csv data set: collezione di 6,000 titles disponibili nel Novembre 2019 su Netflix
- 4 colonne:
 - video's title
 - director
 - data in cui Netflix h aggiunto il contenuto
 - categoria
- colonne director e date_added contengono valori mancanti (ad esempio posizioni 0, 2, and 5836)

0.59 Assignment 5

- Trovare tutte le righe Fi con il titolo "Limitless".
- Trovare tutte le righe con direttore "Robert Rodriguez" e tipo "Movie".
- Trovare tutte le righe con data "2019-07-31" o direttore "Robert Altman".
- Trovare tutte le righe con direttore "Orson Welles", "Aditya Kripalani", o "Sam Raimi".
- Trovare tutte le righe in cui la data è tra May 1, 2019 e June 1, 2019.
- Drop tutte le righe con un valore NaN nella colonna director
- Trovare i giorni in cui Netflix ha aggiunto solo un video al catalogo

[]: